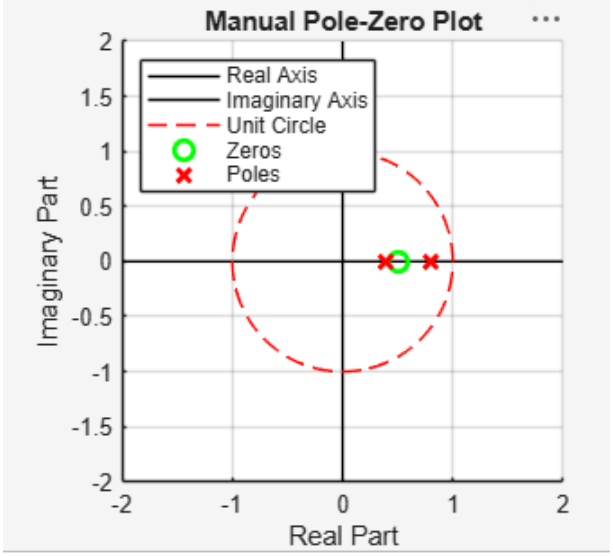
10.TRANSFORM TECHNIQUES AND SYSTEM REPRESENTATION

Experiment 10.1

Pole Zero Plot

```
% Define the coefficients of the
transfer function
b = [1 -0.5];              % Numerator
coefficients (zeros)
a = [1 -1.2 0.32];         % Denominator
coefficients (poles)

% Calculate poles and zeros
z = roots(b);               % Calculate
zeros
p = roots(a);               % Calculate
poles

% Create a new figure
figure;
hold on;
grid on;

% Define real and imaginary axis range
re = -2:0.01:2;
im = -2:0.01:2;

% Plot real axis
plot(re, zeros(size(re)), '-k');

% Plot imaginary axis
plot(zeros(size(im)), im, '-k');

% Plot unit circle
theta = linspace(0, 2*pi, 100);
unit_circle_x = cos(theta);
unit_circle_y = sin(theta);
plot(unit_circle_x, unit_circle_y, '--
r');

% Plot zeros
plot(real(z), imag(z), 'go',
'MarkerSize', 8, 'LineWidth', 2);

% Plot poles
plot(real(p), imag(p), 'rx',
'MarkerSize', 8, 'LineWidth', 2);

% Customize the plot
xlabel('Real Part');
ylabel('Imaginary Part');
```

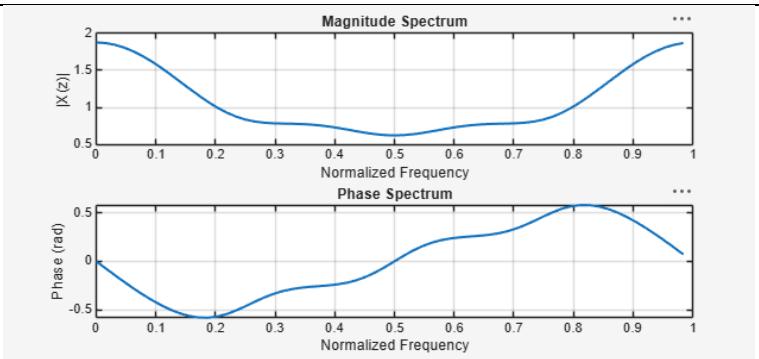


```
title('Manual Pole-Zero Plot');

legend('Real Axis', 'Imaginary Axis',
'Unit Circle', 'Zeros', 'Poles', ...
    'Location', 'northwest');

axis equal;
xlim([-2 2]);
ylim([-2 2]);

hold off;
```

Experiment 10.2

Z-Transform and Inverse Z-Transform

```
% Given sequence
x = [1 0.5 0.25 0.125];

% Number of points for DFT
N = 64;

% Time index
n = 0:length(x)-1;

% Define z-points on unit
circle
k = 0:N-1;
omega = 2*pi*k/N;
z = exp(1i*omega);

% Compute Z-transform on unit
circle (DTFT)
Xz = zeros(1,N);

for m = 1:N
    Xz(m) = sum(x .*
(z(m).^(-n)));
end


figure;

% Magnitude spectrum
subplot(2,1,1);
plot(omega/(2*pi), abs(Xz),
'LineWidth', 1.5);
xlabel('Normalized
Frequency');
ylabel('|X(z)|');
title('Magnitude Spectrum');
grid on;
```



```
% Phase spectrum
subplot(2,1,2);
plot(omega/(2*pi),
atan(imag(Xz)./real(Xz)),
'LineWidth', 1.5);
xlabel('Normalized
Frequency');
ylabel('Phase (rad)');
title('Phase Spectrum');
grid on;


x_rec = zeros(1,N);

for m = 1:N
    x_rec(m) = (1/N) * sum(Xz
.* (z.^n(m)));
end

% Display recovered sequence
disp('Recovered sequence:');
disp(x_rec(1:length(x)));
```